

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

25

Course Code:	CSA1407	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	M. Vinay	Reg. No.:	192311146
Section / Batch:	CSE 2023	Date:	16-07-2026

Code of Conduct

/ M. Vinay (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. Vinay

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	4 / 4	3 / 4	91.25
2	4 / 4	3 / 4	3 / 4	4 / 4	4 / 4	88.75
3	3 / 4	4 / 4	3 / 4	4 / 4	4 / 4	88.75
4	4 / 4	4 / 4	3 / 4	3 / 4	3 / 4	85
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	88.75
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. D. A. A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) $(\epsilon + aa^*)^*$ b) $(a^* + b^*)^* abb$ c) $(a^* b^*)^* aba (a^* + b^*)^*$ d) $(a + ab)^*$ e) $(aba)^* + (a^* b^*)^*$
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of $\{0,1\}^*$: i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1's.

5 Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, :, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:
`int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }`

Questions:

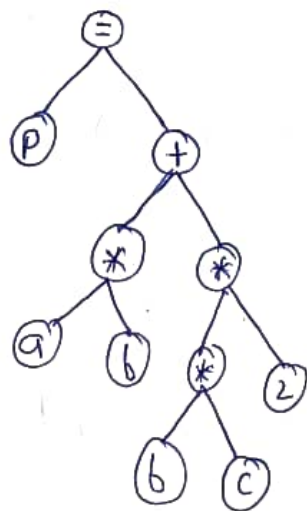
- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

1) $P = (a * b) + (b * c) * 2$

i) Tokens

$\langle id1 \rangle \Rightarrow \langle c \rangle \langle a \rangle \langle * \rangle \langle b \rangle \langle + \rangle \langle c \rangle \langle b \rangle \langle * \rangle$
 $\langle c \rangle \langle \rangle \rangle \langle * \rangle \langle 2 \rangle$

ii) Syntax tree



iii) Intermediate code (TAC)

$$t_1 = b * c$$

$$t_2 = \text{int to real } 2.0$$

$$t_3 = t_1 * 2.0$$

$$t_4 = a * b$$

$$t_5 = t_4 + t_3$$

$$P = t_5$$

iv) Optimization of Intermediate code

$t_1 = b * c$
 $t_2 = t_1 * 2$
 $t_3 = a * b$
 $P = t_2 + t_3$

v) Code Generator

Load R0, b
MUL R0, c
MUL R0, #2
LOAD R1, a
MUL R1, b
ADD R1, R0
STORE P, R1

2) For each of the fig construct the lang write any 6 valid strings

a) $(\epsilon + aa^*)^*$

Accepted string

- ϵ
- a
- aa
- aaaa
- aaaaa

b) $(a^* + b^*)^* abb$

Accepted string

- abb
- aabb
- babb
- aaabb
- bbabb

$(a^* b^*)^* aba (a^* + b^*)^*$

Accepted string

- a b a
- a a b a
- a b a a
- a b a b a
- b a a b a

d) $(a + ab)^*$

Accepted string

- ϵ
- a
- a b
- a a
- a a b

e) $(aba)^* + (a^* b^*)^*$

- ϵ
- a b a
- a b
- a a a b b b
- a b a a a

3) i) Find the number of tokens in following c statement is

```
main()
```

```
{
```

```
int a=10;
```

```
char b="abc";
```

```
int t c=30;
```

```
char d="xyz"
```

```
in /* comment */ m=40.5;
```

```
}
```

S.No	Token	TYPE
1	main	key word
2	(	SeParator
3	)	SeParator
4	{	SeParator
5	int	key word
6	a	Identifiers
7	+	Operator
8	10	constant
9	;	SeParator
10	char	key word
11	b	Identifier
12	=	Operator
13	"abc"	constant
14	;	SeParator
15	int	key word
16	c	Identifier
17	=	Operator
18	30	constant
19	;	SeParator
20	char	key word

21	d	Identifier
22	=	operator
23	"xyz"	constant
24	;	separator
25	int	keyword
26	m	Identifier
27	=	operator
28	40.5	constant
29	;	separator
30	}	separator

ii) Find Lexical error in program

```
void main()
{
    int x=10, y=20;
    char *a;
    a = &x;
    x = 1xab;
}
```

1xab → Invalid literal (lexical error)
 bcz hexadecimal values should begin with
 0x, eg.

0xAB

a = &x: → No lexical error (& is valid operator)

e	Separator
var123	Identify
=	Operator
var456	Identify
=	Operator
var456	Identify
)	Separator
{	"
return	Key Word
;	Separator
}	Separator

c) fsm state transitions

- Keywords $\rightarrow$ 4 transitions
- Identifiers $\rightarrow$ 4 "
- Operators $\rightarrow$ 3 "
- Integer literal $\rightarrow$ 1 "
- Floating-point literal $\rightarrow$ 1 "
- Separators $\rightarrow$ 7 transitions

5) a) How many tokens are there in given input string

1. Int

2 Var123

3 =

4 12345

5 ;

6 var456

7 float

8 =

9 123.45

10 ;

11. if

12. (

13. Var123

14. X =

15. var456

16.)

17. {

18. Return

19. ;

20. }

Total tokens = 20

B) categorize the tokens

Token	Type
Int	keyword
Var123	Identifier
=	operator
12345	integer literal
;	separator
float	keyword
var456	Identifier
=	operator
123.45	Floating-point literal
;	separator
if	keyword

char *a → No lexical error

∴ The lexical error in b line

X = |xab

Invalid Token

4) i) The language of all strings that have an even number of 0's

$(1^* 0 1^* 0 1^*)^* 1^*$

ii) The language of all string S that contain atleast one 1.

$(0|1)^* 1 (0|1)^*$

iii) The language of all strings in which both the no. of 0's & no. of 1's are odd

$(01|10) (00|11)^* (01|10) (01|10)^* (01|10)$

iv) The lang of all strings that start with 1 & end with 0

$1 (0|1)^* 0$

v) The lang of all strings that do not contain consecutive 1's

$(0|10)^* (1|\epsilon)$